

status: draft

SPEC-001: Transcript-Driven Video Editor

Overview

`ar-edit` is a CLI video editor that enables transcript-based editing of video files. Users (human or LLM agent) edit video by manipulating text transcripts rather than scrubbing timelines. The tool transcribes video locally using `whisper.cpp`, maps word-level timestamps to video segments, and renders edits via `ffmpeg`. Preview playback uses `VLC` with timecode and shot ID overlays for precise human-agent feedback.

User Profiles

- [Video Editor](#) — Human user who edits via CLI
- [LLM Agent](#) — AI agent that drives the tool programmatically

Happy Paths

- [Video Editor Happy Paths](#)
- [LLM Agent Happy Paths](#)

Functional Requirements

Project Management

REQ-001: Project Initialization

The system SHALL create a project directory with a configuration file and subdirectories for sources, transcripts, and edits WHEN the user runs `ar-edit init <name>` WITH the result being a valid project structure that subsequent commands can operate on.

Trace:

- TEST-001
- CON-001

REQ-002: Source Registration

The system SHALL register one or more video files as sources in the project, assigning each a stable source ID (format: `src-NNN`) WHEN the user runs `ar-edit add <file>...` WITH the result being source metadata (path, duration, codec, resolution) stored in the project manifest.

Trace:

- TEST-002
- CON-001

REQ-003: Source Validation

The system SHALL validate that each added file is a readable video container with at least one audio stream WHEN registering a source, rejecting files that cannot be processed by ffmpeg WITH a structured error identifying the issue.

Trace:

- TEST-003
-

Transcription

REQ-004: Local Transcription via whisper.cpp

The system SHALL transcribe a registered source's audio using whisper.cpp locally, producing a transcript with word-level timestamps WHEN the user runs `ar-edit transcribe <source-id>` or `ar-edit transcribe --all` WITH the result being a JSON transcript file stored in the project's transcripts directory.

Trace:

- TEST-004
- CON-002

REQ-005: Transcript JSON Format

The system SHALL produce transcript files in a JSON format containing: source ID, total duration, an array of segments (each with start/end time and text), and within each segment an array of words (each with start time, end time, word text, and sequential word index) WITH word timestamps having millisecond precision.

Trace:

- TEST-005
- CON-002

REQ-006: Whisper Model Selection

The system SHALL allow the user to specify a whisper.cpp model (tiny, base, small, medium, large) via `--model <name>` flag, defaulting to `base` WHEN no model is specified.

Trace:

- TEST-006

REQ-007: Pre-existing Transcript Import

The system SHALL accept pre-existing transcript files (SRT, VTT, or whisper.cpp JSON) via `ar-edit transcribe <source-id> --import <file>`, converting them to the internal JSON format WITH word-level timestamps preserved where available.

Trace:

- TEST-007
 - CON-002
-

Transcript Operations

REQ-008: Transcript Listing

The system SHALL list all transcribed sources with summary metadata (source ID, duration, word count, file path) WHEN the user runs `ar-edit transcripts list` WITH output in human-readable table or `--json` format.

Trace:

- TEST-008
- CON-003

REQ-009: Transcript Reading

The system SHALL output a full transcript WHEN the user runs `ar-edit transcripts read <source-id>`, supporting `--json` for structured output and plain text for human reading.

Trace:

- TEST-009
- CON-003

REQ-010: Transcript Search

The system SHALL search across all transcripts for a text query, returning matching segments with source ID, word index range, timestamps, and surrounding context WHEN the user runs `ar-edit transcripts search <query>` WITH output in human-readable or `--json` format.

Trace:

- TEST-010
 - CON-003
-

Source Index and Search

REQ-031: Source Indexing

The system SHALL build a searchable index for each registered source containing: (a) technical metadata (codec, resolution, frame rate, duration, audio channels, bitrate), (b) transcript text and word timestamps, and (c) keyframe thumbnails extracted at configurable intervals (default: every 10 seconds) WHEN the user runs `ar-edit index <source-id>` or `ar-edit index --all` WITH the index stored as structured JSON in the project directory.

Trace:

- TEST-037
- CON-008

REQ-032: Keyframe Thumbnail Extraction

The system SHALL extract representative frame images (JPEG, 640px wide) from each source at scene changes and/or fixed intervals WHEN indexing, storing them in a `thumbnails/` subdirectory with filenames encoding source ID and timestamp (e.g. `src-001_00m30s.jpg`) WITH the thumbnail paths referenced in the index.

Trace:

- TEST-038

REQ-033: Visual Scene Description

The system SHALL generate a text description file for each extracted thumbnail suitable for LLM consumption WHEN the user runs `ar-edit index <source-id> --describe` WITH the descriptions stored as a JSON array (thumbnail path, timestamp, description placeholder) that can be populated by an external tool or LLM agent. The system itself does NOT run vision models — it produces the scaffolding for an agent to fill in.

Trace:

- TEST-039
- CON-008

REQ-034: Index Search

The system SHALL search across the full index (metadata, transcripts, and scene descriptions) for a text query WHEN the user runs `ar-edit search <query>` WITH results ranked by relevance, returning source ID, timestamp, match type (transcript/metadata/scene), context snippet, and word indices where applicable, in human-readable or `--json` format.

Trace:

- TEST-040
- CON-008

REQ-035: Index Summary

The system SHALL produce a per-source summary (total duration, scene count, key topics from transcript, thumbnail count) WHEN the user runs `ar-edit index show <source-id>` WITH output in human-readable or `--json` format.

Trace:

- TEST-041
- CON-008

Edit Document

REQ-011: Edit Document Creation

The system SHALL create a new empty edit document WHEN the user runs `ar-edit edit create <name>` WITH the result being a JSON file in the project's edits directory.

Trace:

- TEST-011
- CON-004

REQ-012: Segment Addition

The system SHALL append a segment to an edit document, referencing a source ID and a range, WHEN the user runs one of:

- `ar-edit edit add-segment <edit> --source <id> --from-word <N> --to-word <M>` (word range, requires transcript)
- `ar-edit edit add-segment <edit> --source <id> --from-scene <N> --to-scene <M>` (scene range, requires index)
- `ar-edit edit add-segment <edit> --source <id> --from-ms <N> --to-ms <M>` (time range, always works)

WITH the segment receiving a sequential shot ID (format: `shot-NNN`). Exactly one range type must be specified.

Trace:

- TEST-012
- CON-004

REQ-013: Segment Reordering

The system SHALL reorder segments within an edit document WHEN the user runs `ar-edit edit move-segment <edit> --shot <shot-id> --position <N>` WITH the result being the segment moved to the specified position (0-indexed) and all shot IDs remaining stable.

Trace:

- TEST-013
- CON-004

REQ-014: Segment Removal

The system SHALL remove a segment from an edit document WHEN the user runs `ar-edit edit remove-segment <edit> --shot <shot-id>` WITH the shot ID retired (not reused).

Trace:

- TEST-014
- CON-004

REQ-015: Segment Trimming

The system SHALL modify the word range of an existing segment WHEN the user runs `ar-edit edit trim-segment <edit> --shot <shot-id> --from-word <N> --to-word <M>` WITH the result updating the segment boundaries.

Trace:

- TEST-015
- CON-004

REQ-016: Edit Document Display

The system SHALL display the contents of an edit document as a table of segments (shot ID, source ID, word range, time range, text preview, duration) WHEN the user runs `ar-edit edit show <edit>` WITH output in human-readable or `--json` format.

Trace:

- TEST-016
- CON-004

REQ-017: Edit Document Validation

The system SHALL validate an edit document, checking that all source references exist, all word indices are within transcript bounds, and no segment has zero duration WHEN the user runs `ar-edit validate <edit>` WITH structured output reporting all errors found.

Trace:

- TEST-017
- CON-005

REQ-018: Edit Document from Annotated Transcript

The system SHALL parse an annotated markdown transcript file (exported via REQ-020), resolve text selections back to source/word-index pairs, and produce an edit document WHEN the user runs `ar-edit edit from-transcript <file>` WITH errors reported for unresolvable text.

Trace:

- TEST-018
 - CON-004
-

Transcript Export for Text Editing

REQ-019: Direct Edit Document Authoring

The system SHALL accept a hand-written or agent-written edit document in JSON format, validating its schema before use WITH the JSON schema documented and available via `ar-edit schema edit`.

Trace:

- TEST-019
- CON-004

REQ-020: Editable Transcript Export

The system SHALL export all project transcripts as a single markdown document with embedded source/word-index annotations (as HTML comments) WHEN the user runs `ar-edit transcripts export --format editable` WITH the format designed to survive text editing (deletion, reordering) while preserving annotation integrity.

Trace:

- TEST-020
-

Preview and Playback

REQ-021: Segment Preview

The system SHALL open a video player (VLC preferred, ffmpeg fallback) playing a specific segment from an edit document WHEN the user runs `ar-edit preview <edit> --shot <shot-id>` WITH playback starting at the segment's start time and stopping at its end time.

Trace:

- TEST-021
- CON-006

REQ-022: Full Edit Playback

The system SHALL render a full preview of the edit and play it in the video player WHEN the user runs `ar-edit play <edit>` WITH playback proceeding through all shots in sequence. This is the primary review mode — the user watches the assembled edit as a continuous video.

Trace:

- TEST-022
- CON-006

REQ-023: Timecode and Shot ID Overlay (toggle)

The system SHALL support an overlay mode that burns onto playback: (a) running timecode in HH:MM:SS.mmm format, (b) the current shot ID (e.g. `shot-003`), (c) the source ID (e.g. `src-002`), and (d) scene description or transcript snippet for the current segment WHEN the user passes `--overlay` to the play command. The overlay SHALL be toggleable:

- `ar-edit play <edit>` — clean playback, no overlay
- `ar-edit play <edit> --overlay` — full overlay (timecode + shot ID + source)
- `ar-edit play <edit> --overlay minimal` — timecode only

The overlay SHALL use a semi-transparent background bar in the top-left corner, monospace font, and SHALL NOT be present in final renders unless the user passes `--burn-overlay` to the render command.

REQ-037: Playback Pause-and-Feedback Workflow

The system SHALL, after playback completes or is interrupted, output a structured summary of the last-viewed position including: (a) the shot ID visible at the pause point, (b) the timecode, (c) the source ID, and (d) the word index or scene index at that position WHEN `--json` is passed WITH the output designed for an LLM agent to receive as context for editing instructions (e.g. user says "on shot-005, cut 3 seconds earlier" and the agent has all the data to act on it).

Trace:

- TEST-023
- CON-006
- OBS-001

REQ-024: Source Video Playback

The system SHALL open a source video at a specific timestamp WHEN the user runs `ar-edit play <source-id> --at <timecode>` or `ar-edit play <source-id> --at-word <N>` WITH the player opening at the resolved timestamp.

Trace:

- TEST-024

Rendering

REQ-025: Final Render

The system SHALL render an edit document to a video file by: extracting each segment from its source at word-boundary timestamps, concatenating segments in order, and encoding to the output format WHEN the user runs `ar-edit render <edit> -o <output>` WITH the render using ffmpeg for all encoding operations.

Trace:

- TEST-025
- CON-007

REQ-026: Render Format Options

The system SHALL accept encoding options (codec, resolution, bitrate, format) via CLI flags or project defaults WHEN rendering, defaulting to H.264/AAC in MP4 container matching the highest input resolution.

Trace:

- TEST-026
- CON-007

REQ-027: Render Progress Reporting

The system SHALL report render progress as structured output (percentage, current segment, ETA) WHEN `--json` flag is present, and as a progress bar WHEN in human-readable mode.

Trace:

- TEST-027
 - OBS-001
-

REQ-036: Subtitle Track Embedding

The system SHALL embed an SRT subtitle track derived from the transcript segments in the rendered output WHEN the user passes `--subtitles` to the render command WITH subtitle timing matching the word-level timestamps of the edit document.

Trace:

- TEST-042
 - CON-007
-

Agent Interface

REQ-028: Structured JSON Output

The system SHALL output structured JSON for every command WHEN the `--json` flag is present, including: success/error status, result data, and error details with actionable context FOR the LLM Agent user.

Trace:

- TEST-028
- CON-003

REQ-029: Idempotent Operations

The system SHALL make all read operations (list, show, read, search, validate) idempotent and side-effect-free WITH no mutation of project state.

Trace:

- TEST-029

REQ-030: Exit Codes

The system SHALL use conventional exit codes: 0 for success, 1 for user error (bad input), 2 for system error (ffmpeg failure, disk full), 3 for validation error WITH exit codes documented and stable across versions.

Trace:

- TEST-030

Non-Functional Requirements

NFR-001: Transcription Performance

Transcription throughput SHALL be at minimum 1x real-time on the `base` model on a 2020-era laptop (M1 MacBook Air or equivalent) WITH the user informed of estimated time before transcription begins.

Trace:

- TEST-031
- OBS-002

NFR-002: Render Performance

Rendering SHALL complete within 3x the output duration for concatenation-only edits (no re-encoding of matching codecs) UNDER standard hardware WITH progress reporting per REQ-027.

Trace:

- TEST-032
- OBS-002

NFR-003: Preview Latency

Preview generation for a single segment SHALL complete in less than 5 seconds for segments under 30 seconds UNDER standard hardware WITH the player opening within 2 seconds of preview readiness.

Trace:

- TEST-033

NFR-004: Transcript Accuracy

The system SHALL preserve all word-level timestamps from whisper.cpp without modification or rounding WITH timestamp precision of 10ms or better.

Trace:

- TEST-034

NFR-005: Project Portability

Project directories SHALL be self-contained (all paths relative) and copyable between machines WITH only external dependencies being ffmpeg, whisper.cpp, and a video player.

Trace:

- TEST-035

NFR-006: Edit Document Size

The system SHALL handle edit documents with up to 500 segments across up to 50 sources WITHOUT degradation in validation or preview generation time.

Trace:

- TEST-036

Ambiguity Log

The following terms have been resolved with measurable criteria:

Original Term	Resolution
"fast preview"	Preview single segment < 5s, player opens < 2s after (NFR-003)
"quick render"	Concat-only render < 3x output duration (NFR-002)
"word boundary precision"	Timestamps preserved at 10ms precision from whisper.cpp (NFR-004)
"readable overlay"	Semi-transparent background, top-left position, monospace font (REQ-023)
"large project"	Up to 500 segments, 50 sources (NFR-006)

Resolved Decisions

#	Question	Decision
1	Transition effects in MVP?	Hard cuts only. Transitions deferred to future release.
2	Audio normalization?	No normalization. Audio levels left as-is; user handles in post.
3	Subtitle track in render?	Yes, via <code>--subtitles</code> flag. Embeds SRT track in output MP4.
4	whisper.cpp vs Python whisper?	whisper.cpp only. Faster, no Python/PyTorch dependency.
5	Implementation language?	Rust. Single binary distribution, fast execution.

Technology Decisions (Preliminary — to be formalized as ADRs in Phase 2)

Concern	Decision	Rationale
Implementation language	Rust	Single binary distribution, fast execution, strong CLI ecosystem
Terminal UI	<code>ratatui</code>	Interactive TUI for human editing; the agent bypasses this via <code>--json</code> CLI mode
Transcription engine	whisper.cpp CLI (subprocess)	Local, fast, word-level timestamps via <code>--output-json</code>
Video processing	ffmpeg CLI (subprocess)	Universal, battle-tested, handles all codecs
Thumbnail extraction	ffmpeg <code>-vf thumbnail</code> / scene detect	Built into ffmpeg, no additional deps

Concern	Decision	Rationale
Preview playback	VLC (via <code>cvlc</code> / <code>vlc</code> CLI), ffplay fallback	VLC is ubiquitous, supports seeking; ffplay is always available with ffmpeg
Overlay rendering	ffmpeg drawtext filter	Built into ffmpeg, no additional deps
CLI framework	<code>clap</code> (Rust)	Derive macros, subcommands, shell completions, well-maintained
JSON handling	<code>serde</code> + <code>serde_json</code> (Rust)	De facto standard, excellent performance
Edit document format	JSON	Agent-friendly, schema-validatable, human-readable enough
Transcript storage	JSON with word-level timestamps	Matches whisper.cpp <code>--output-json</code> format, easily queryable
Project manifest	JSON	Consistent with edit documents and transcripts
Search / indexing	<code>tantivy</code> (Rust) or simple in-memory grep	Full-text search across transcripts and descriptions